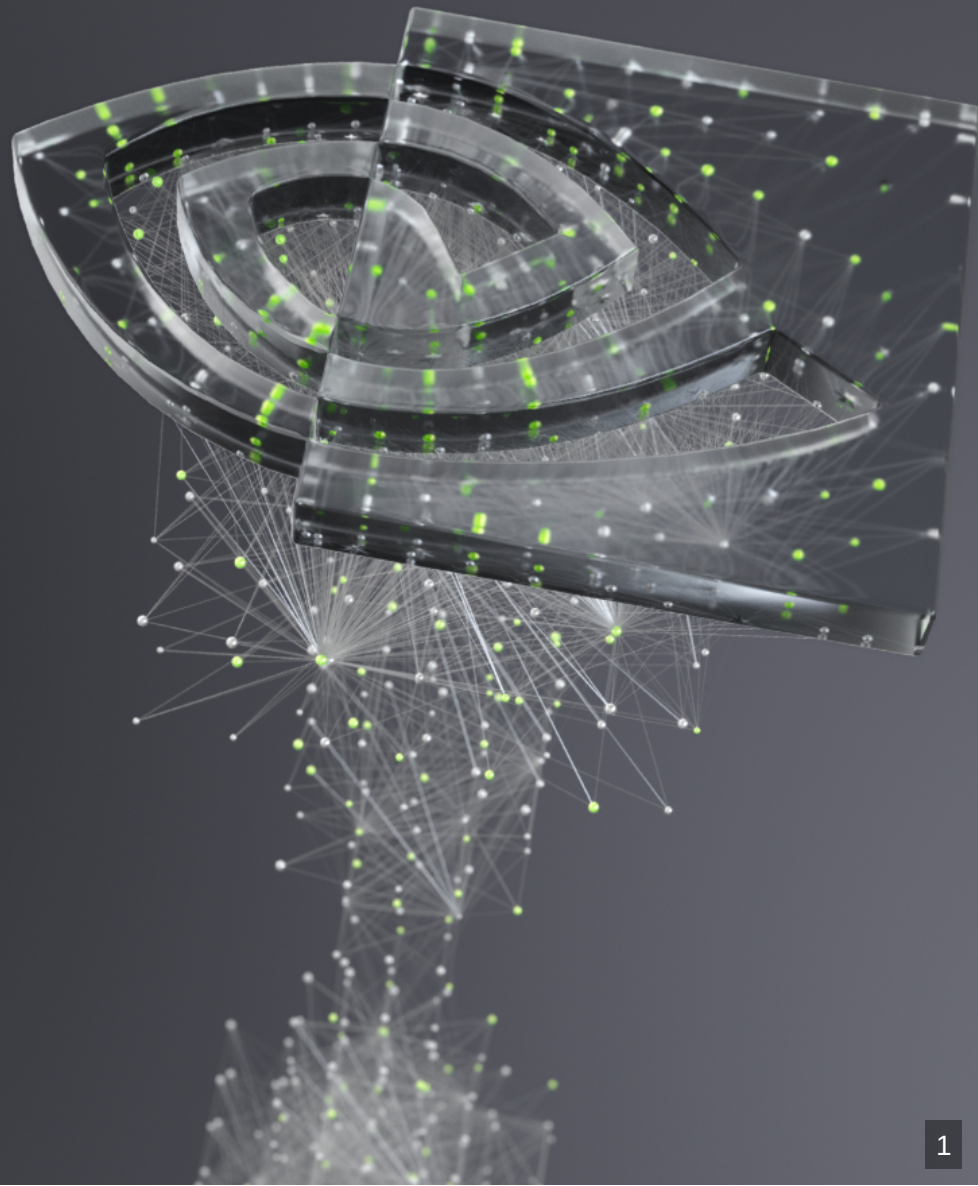
NVIDIA

TC PERFORMANCE UPDATE



NETDEV 0X14 AUGUST 2020

VLAD BUSLOV

VLADBU@NVIDIA.COM

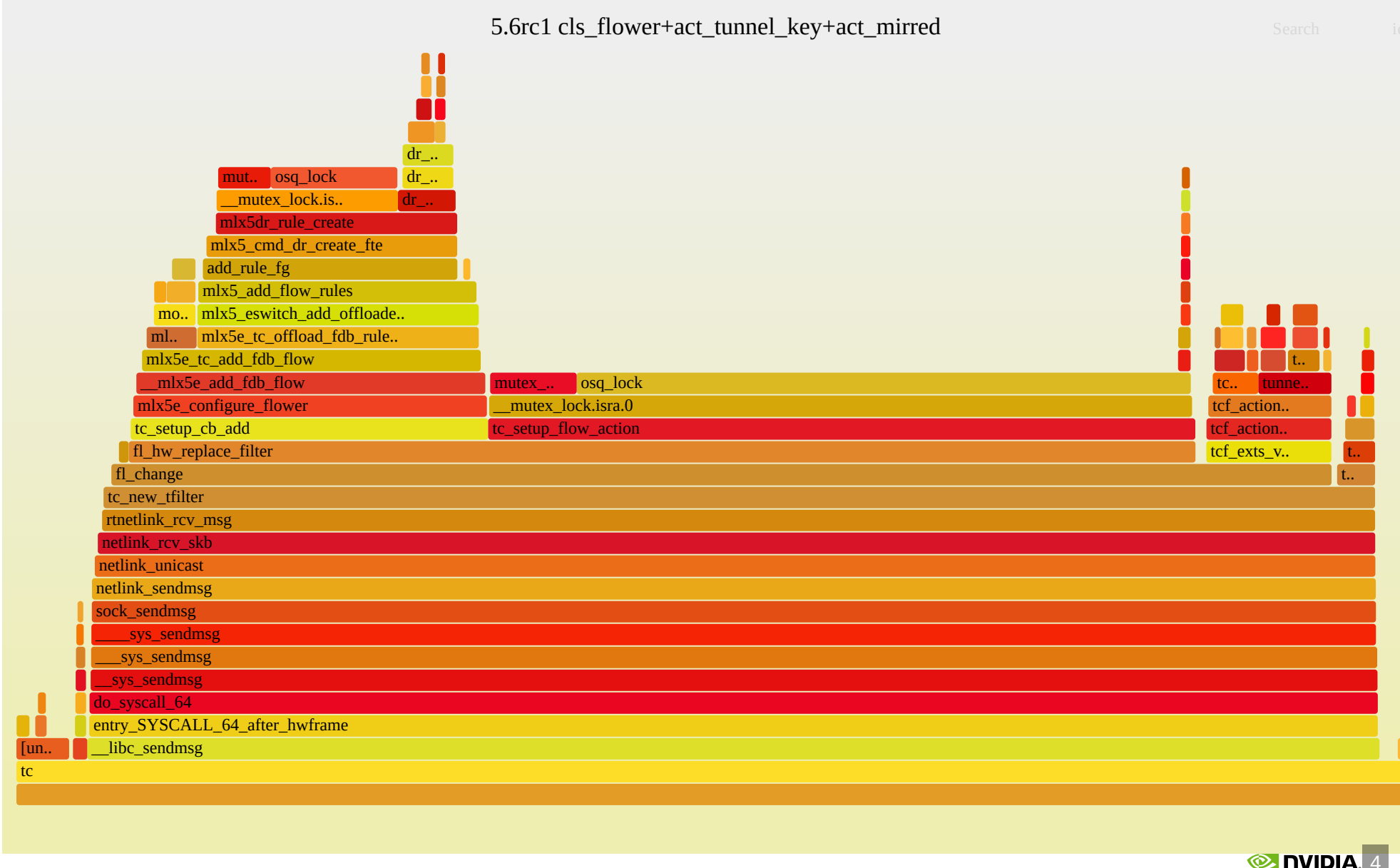
QUICK RECAP

- Cls_api filter update path rtnl lock dependency removed.
- Cls_flower converted to fine-grained locking.
- All actions converted to fine-grained locking.
- Drivers can signal to cls API that they do not require rtnl_lock with flow_block_offload->unlocked_driver_cb flag in ndo_setup_tc().

INTERACTION BETWEEN TC FILTER ADD AND DUMP

- Prior to 5.7 flow_action infrastructure code briefly obtained rtnl in tc_setup_flow_action().
- This didn't result lock contention when just adding rules.
- However, tc filter dump requires rtnl lock synchronization for whole execution.
- Dump holds rtnl until it fills netlink packet!
- Virtual switches concurrently add new flows and dump all flows on attached ports.
- Adding new filters concurrently with dump causes lock contention.

CONCURRENT TC FILTER ADD/SHOW



REMOVE RTNL LOCK DEPENDENCY FROM FLOW_ACTION INFRASTRUCTURE

- All actions had been updated before to use per-action fine-grained locks.
- Just use tc_action->tcfa_lock instead of global rtnl lock.

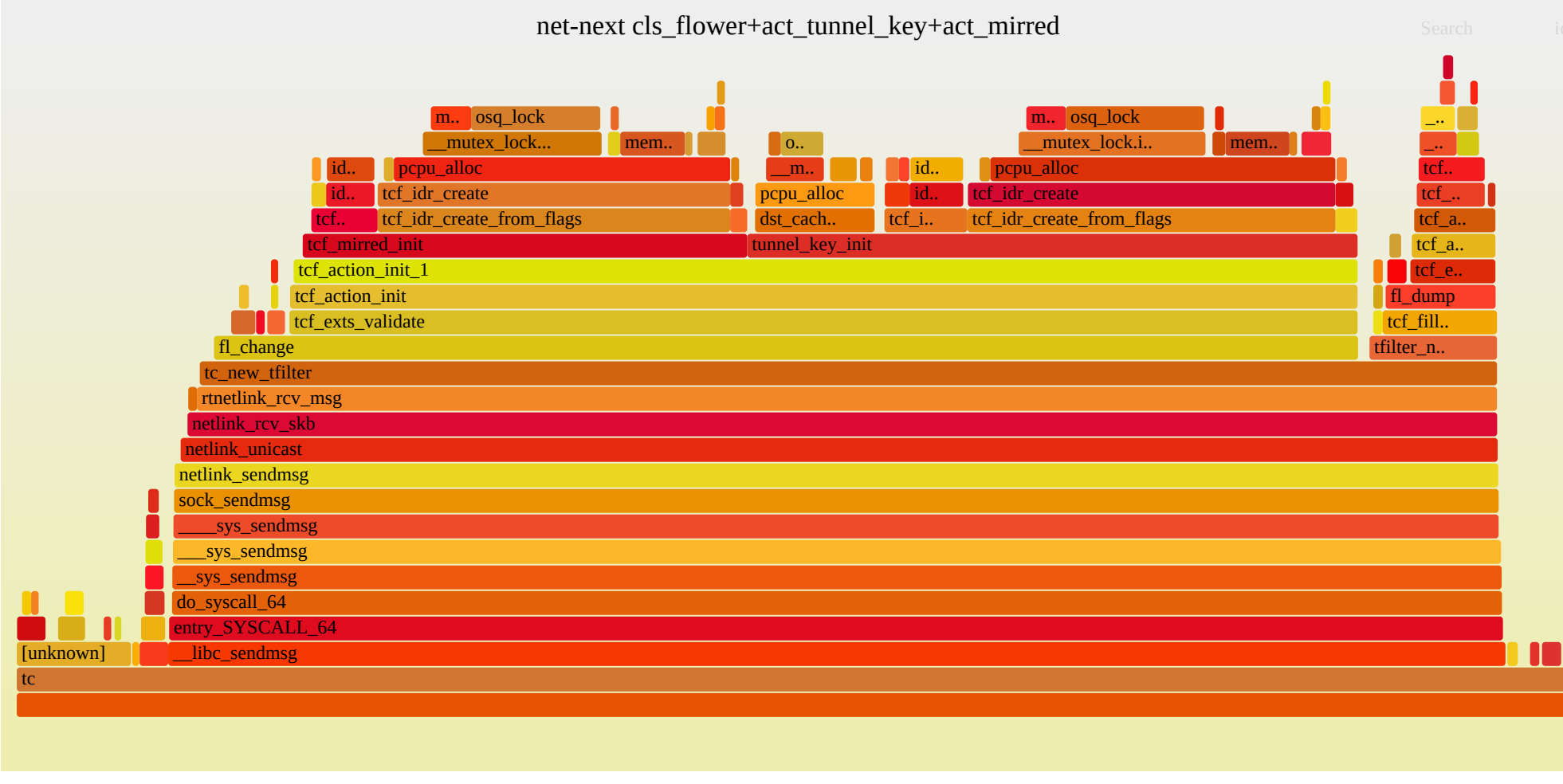
LOCK REMOVAL IMPACT

- "tc filter add" tasks are no longer impacted by "tc filter show" tasks.
- TC rule update no longer competes for rtnl lock with its other users (drivers, etc.).

PERCPU ALLOCATOR PERFORMANCE IN TC ACTIONS

- Percpu allocator in synchronized with single global mutex internally.
- Memory consumption grows linearly with number of CPUs in system.
- Great for data path performance, no so great for control path and memory usage.

TC FILTER ADD WITH PERCPU ACTION STATS



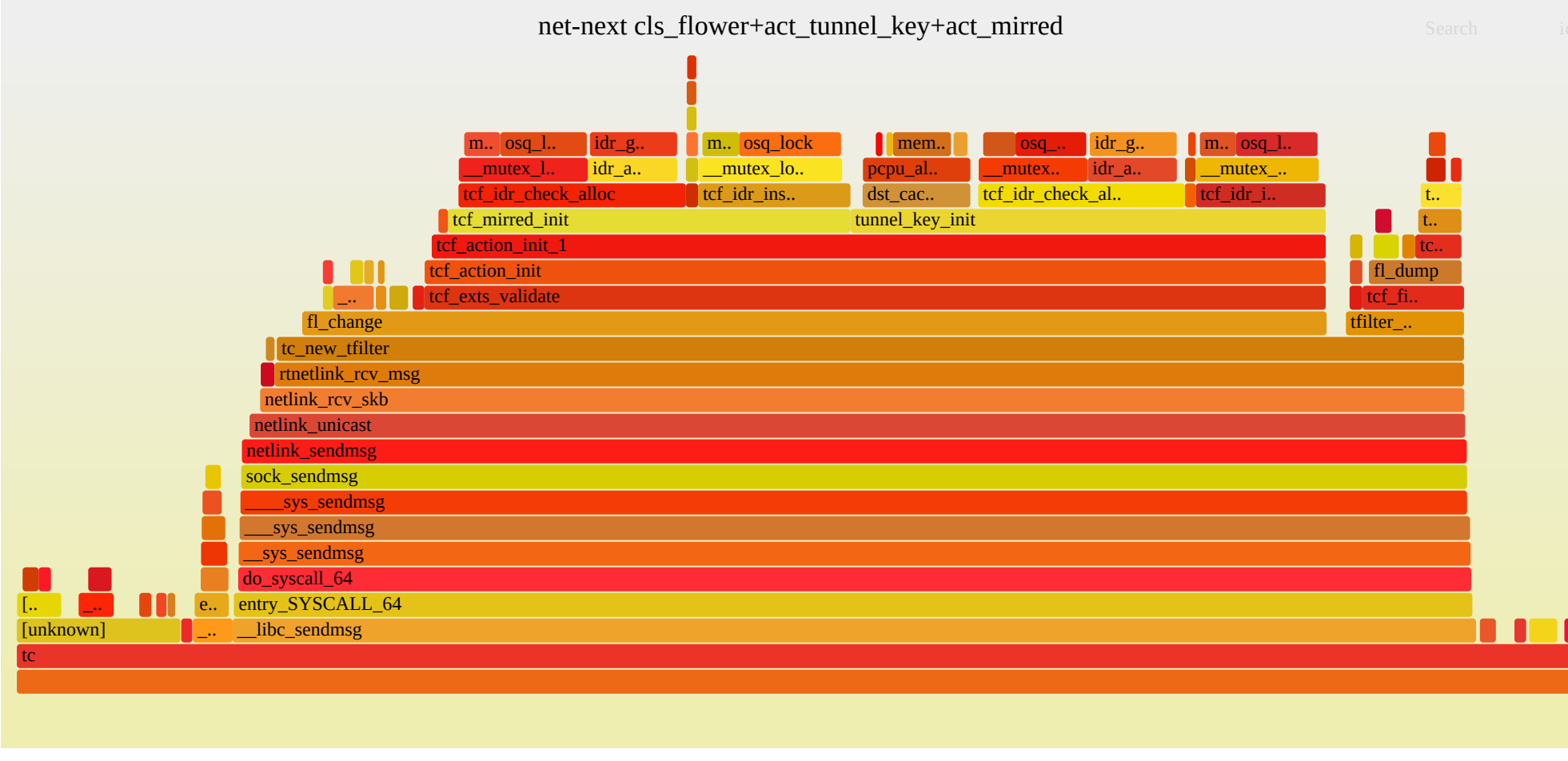
ALLOW USE TO CHOSE THE STATS IMPLEMENTATION MECHANISM

- Even though percpu stats is great for SW data-path performance, there is no benefit when rules are offloaded to HW.
- Make stats implementation mechanism user-configurable with netlink flag (TCA_ACT_FLAGS -> TCA_ACT_FLAGS_NO_PERCPU_STATS).
- When user requests to not use percpu stats, use regular integers synchronized with per-action lock.
- The infrastructure for such approach exists - some actions (pedit) haven't been converted to percpu allocation.

IMPACT OF REMOVING PERCPU STATS

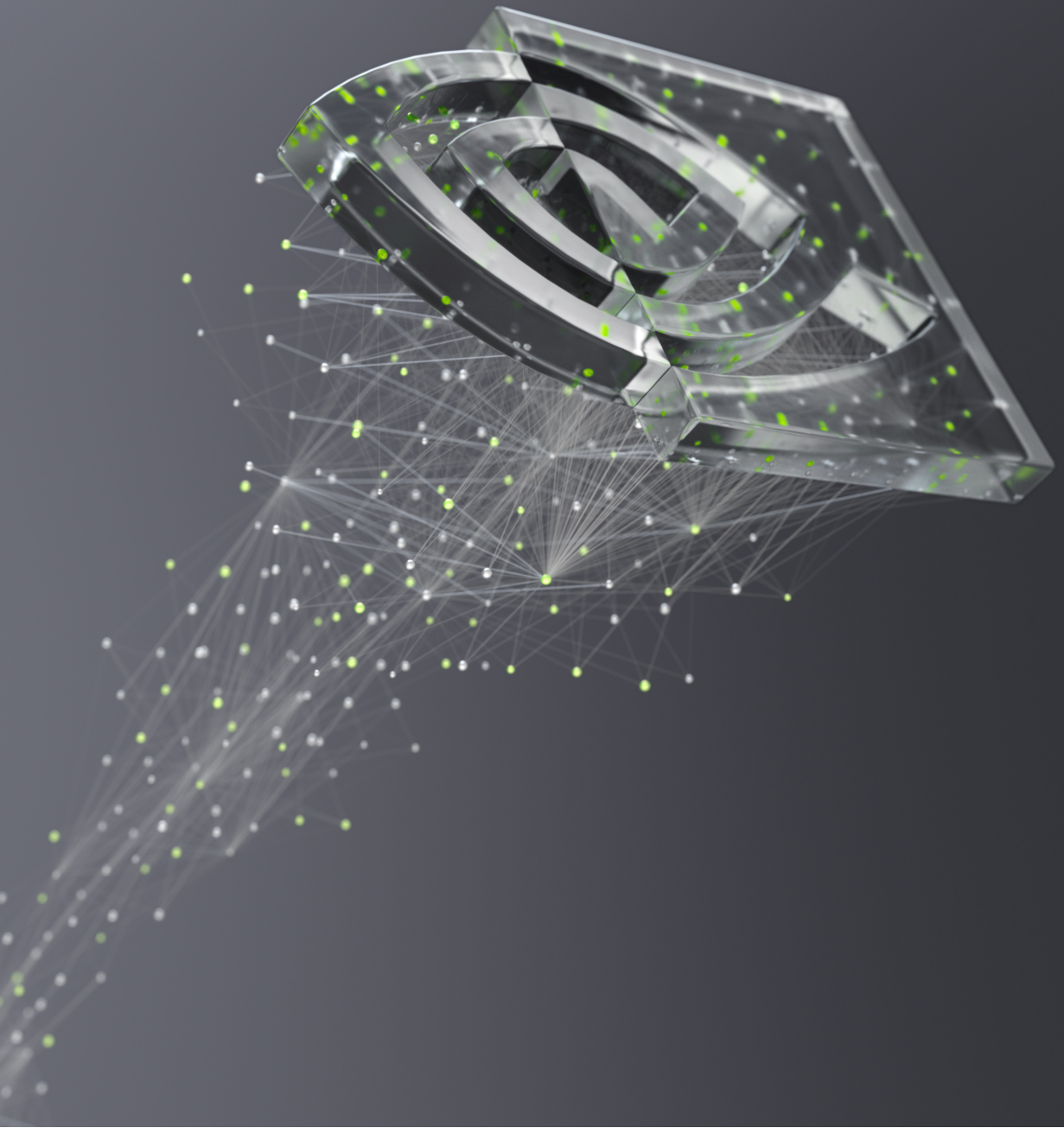
- Significant rules insertion rate increase (30-120% in my tests).
- Improved memory usage (2 * sizeof(struct gnet_stats_basic_cpu) + sizeof(struct gnet_stats_queue) = 52bytes per CPU).

CURRENT NET-NEXT PERFORMANCE



BOTTLENECKS

- Idr becomes bottleneck.
- Percpu allocator still used in some actions.
- tfilter_notify() consumes ~9% CPU.



THANKS!